

Operaciones de Aprendizaje Automático III

Fine-Tuning de Small Language Models

MSc. Fernando Silva



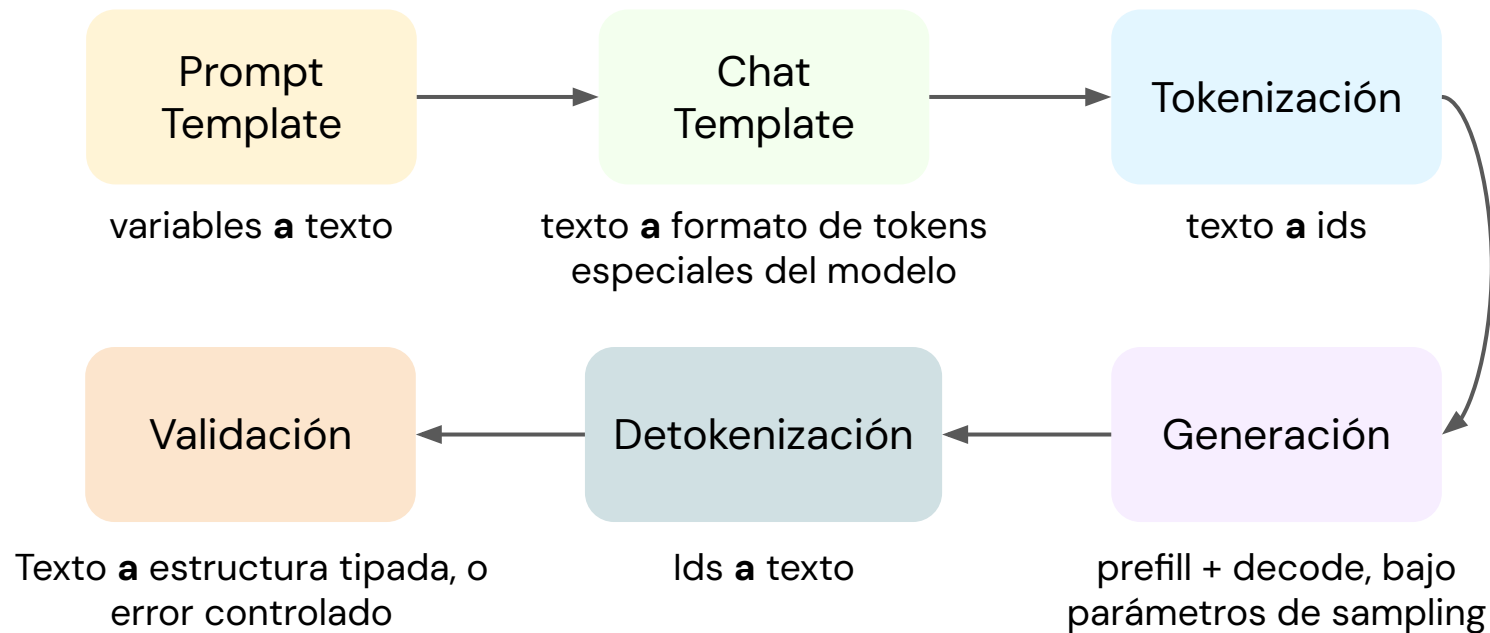
Agenda

- Repaso
- Model Registries y Versionado
- Introducción a Small Language Models
- Instruction tuning
- PEFT
- Fine tuning
- LoRA y QLoRA

Repaso

Anatomía de Inference Pipeline

Todo lo que ocurre entre el input del usuario y el output de nuestra aplicación:



Cada uno de estos seis pasos es un artefacto versionable y un punto de falla independiente.

Model Registries y Versionado

Registro en LLMOps

- En MLOps clásico registramos un modelo entrenado.
- En LLMOps registramos una configuración:
 - Los pesos son una referencia dentro de esa configuración, no el objeto registrado.

Dos versiones del sistema pueden apuntar al mismo modelo y diferir solo en el prompt.

Esas dos versiones son tan distintas entre sí como dos modelos distintos, y hay que poder distinguirlas, compararlas y revertirlas igual.

Registro en LLMOps

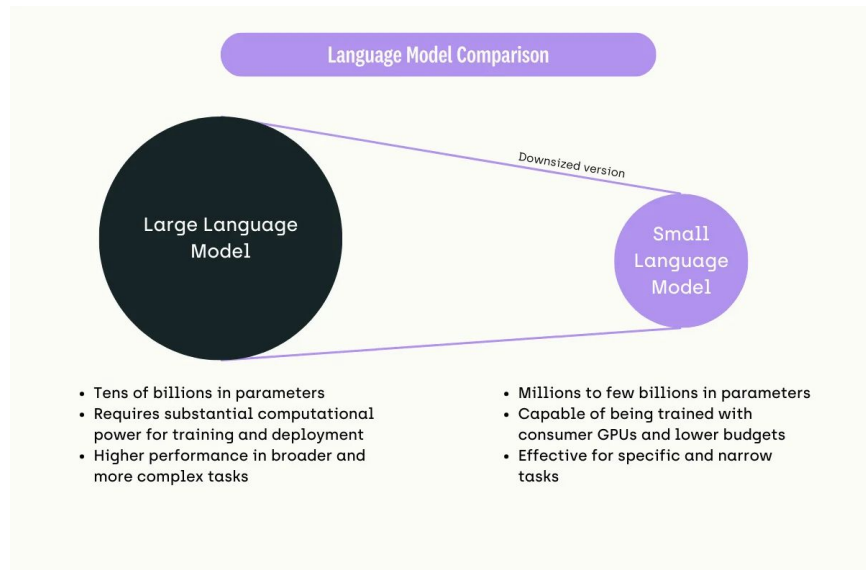
Campo del artefacto	Cómo se registra
1. Modelo y revisión	Parámetro: repo_id y commit SHA completo
2. Tokenizer	Parámetro: repositorio y revisión
3. Chat template	Artefacto: el string
4. Prompt template	Prompt registry, con versión propia
5. Sampling	Parámetros: temperature, top_p, max_tokens, seed y los defaults efectivos que vinieron de generation_config.json
6. Runtime y hardware	Artefacto: entorno.json
7. Recuperación	Parámetros: índice, embeddings, top_k, chunking
8. Evaluación	Métricas y el dataset como artefacto versionado



Introducción a Small Language Models

Small Language Models

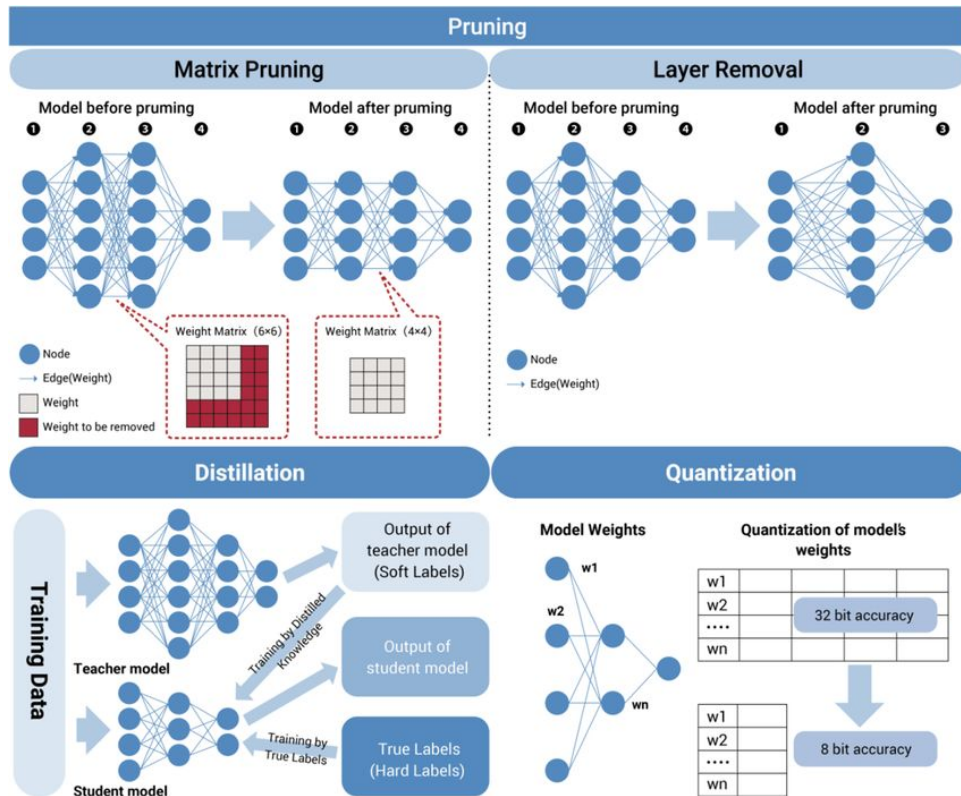
- Los Small Language Models (Modelos de Lenguaje Pequeños) son modelos de lenguaje ligeros, diseñados para funcionar eficientemente en entornos con recursos limitados, como teléfonos inteligentes, sistemas embebidos y computadoras de bajo consumo.
- Mientras que los modelos de lenguaje grandes pueden tener miles de millones o incluso billones de parámetros, los SLM suelen tener entre 1 millón y 10 mil millones de parámetros, manteniendo capacidades fundamentales de NLP como generación de texto, resumen, traducción y respuesta a preguntas.



Small Language Models

El proceso de reducir el tamaño de un modelo de lenguaje implica varias técnicas destinadas a disminuir su tamaño sin comprometer demasiado su rendimiento:

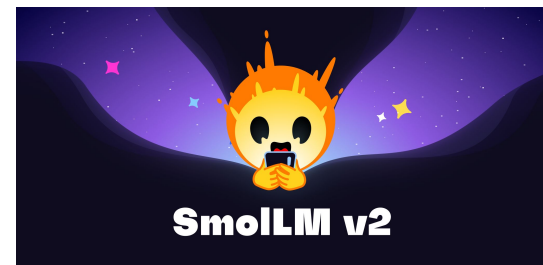
- **Knowledge Distillation:** Entrenar un modelo pequeño, llamado “estudiante”, utilizando el conocimiento transferido de un modelo más grande, llamado “maestro”.
- **Pruning:** Eliminar parámetros redundantes o poco importantes dentro de la arquitectura de la red neuronal.
- **Quantization:** Reducir la precisión numérica utilizada en los cálculos, por ejemplo, convirtiendo valores de punto flotante en valores enteros.



Small Language Models

Hay varios modelos de lenguaje pequeños pero potentes. A continuación, se presentan algunos SLM de entre 1 y 4 mil millones de parámetros:

- **Llama 3.2-1B:** Modelo desarrollado por Meta con 1.000 millones de parámetros, optimizado para dispositivos edge.
- **Qwen 2.5-1.5B:** Modelo de Alibaba diseñado para aplicaciones multilingües, con 1.500 millones de parámetros.
- **DeepSeek-R1-1.5B:** Primera generación de modelos de razonamiento de DeepSeek, destilado a partir de Qwen2.5, con 1.500 millones de parámetros.
- **SmolLM2-1.7B:** Desarrollado por Hugging Face, es un modelo de lenguaje pequeño de última generación con 1.700 millones de parámetros, entrenado con conjuntos de datos especializados.
- **Gemma 3-4B:** Desarrollado por Google DeepMind, es un modelo ligero de 4.000 millones de parámetros, con capacidades multilingües y multimodales.



Aplicaciones

- **Chatbots y asistentes virtuales:** Son lo suficientemente eficientes para ejecutarse en dispositivos móviles y ofrecer interacción en tiempo real.
- **Traducción de idiomas:** Los modelos ligeros pueden ofrecer traducción directamente en el dispositivo, por ejemplo, para viajeros.
- **Resumen y generación de contenido:** Las empresas pueden utilizar SLM para generar textos de marketing, publicaciones en redes sociales e informes.
- **Aplicaciones en salud:** Permiten ejecutar sistemas de IA directamente en el dispositivo para tareas como orientación sobre síntomas o apoyo a la investigación médica.
- **IoT y computación en el borde (Edge Computing):** Pueden ejecutar IA en dispositivos inteligentes sin depender de servicios en la nube.
- **Herramientas educativas:** Los sistemas de tutoría pueden utilizar SLM para generar explicaciones personalizadas, cuestionarios y retroalimentación en tiempo real.

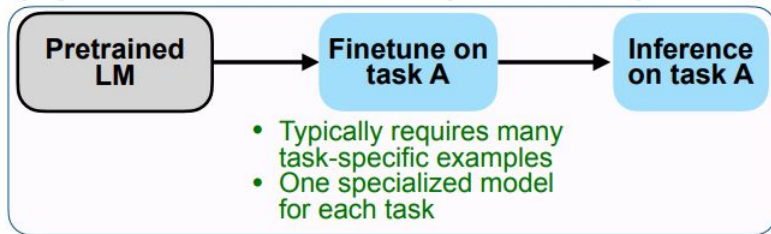
Instruction Tuning

Instruction Tuning

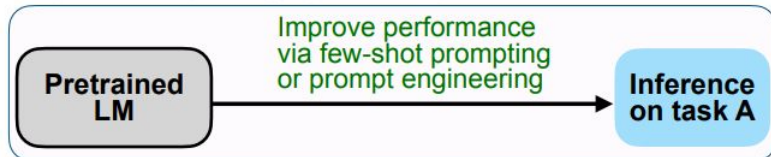
Un LLM preentrenado solo predice el siguiente token: es un modelo de lenguaje, no un modelo asistente.

- Ej: Dado el prompt "Explica la fotosíntesis", un modelo base puede continuar con otra pregunta en vez de responder.
- Necesitamos alinear el formato de interacción (**instrucción: respuesta útil**) sin perder el conocimiento adquirido en pre entrenamiento.

(A) Pretrain–finetune (BERT, T5)

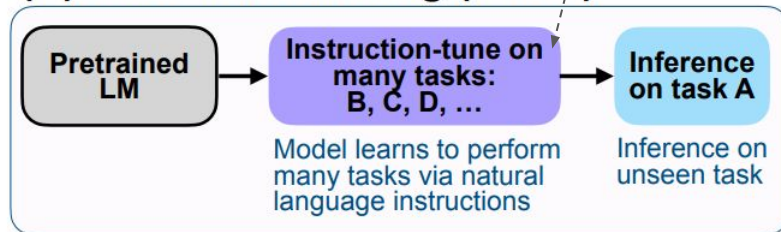


(B) Prompting (GPT-3)



Dataset de pares (instrucción, respuesta)

(C) Instruction tuning (FLAN)



Instruction Tuning

Datasets

- **Generados por humanos:**
 - FLAN, Natural Instructions, Dolly.
- **Generados sintéticamente (self-instruct):**
 - Alpaca (generado con GPT-3)
 - Vicuña (conversaciones de ShareGPT)
 - Orca (explicaciones tipo "razonamiento paso a paso" destiladas de GPT-4).
- **Multi-task instruction datasets:** FLAN Collection agrega +1800 tareas con múltiples formulaciones de instrucción por tarea.

Premise

Russian cosmonaut Valery Polyakov set the record for the longest amount of time spent in space.

Hypothesis

Russians hold the record for the longest stay in space.

Target

Entailment
Not entailment



Options:

- yes
- no

Template 1

Russian Cosmonaut Valery Polyakov set the record for the longest amount of time spent in space.

Based on the paragraph above, can we conclude that

Russians hold the record for the longest stay in space?

OPTIONS

- yes
- no

Template 2

Read the following and determine if the hypothesis can be inferred from the premise:

Premise: **<premise>**

Hypothesis: **<hypothesis>**

<options>

Template 3, ...

Instruction Tuning

Riesgos y desafíos operativos

- **Catastrophic forgetting:** el modelo pierde conocimiento general al sobreajustarse al formato/dominio del dataset de instrucciones.
- **Overfitting al formato:** el modelo aprende "cómo responder" pero memoriza patrones superficiales del dataset (longitud, estilo) en vez de generalizar.
- **Sesgo del dataset:** si las instrucciones sintéticas vienen de otro LLM, se heredan sus sesgos y errores ("model collapse" en cadenas largas de distilación).

Ejemplos de industria:

InstructGPT (OpenAI)

Primer paper que popularizó el pipeline SFT a RM a RLHF.

FLAN-T5 (Google)

Instruction tuning multi-tarea a gran escala, foco en generalización zero/few-shot.

Alpaca (Stanford)

Instruction tuning de LLaMA con 52k instrucciones generadas por GPT-3 (self-instruct), demostrando que se puede lograr comportamiento "chat" con bajo costo.

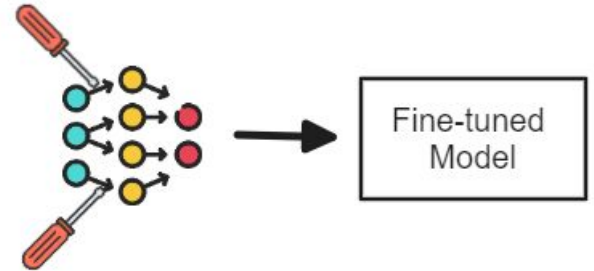
LLaMA-2-Chat / LLaMA-3-Instruct (Meta)

Pipeline combinado de SFT y RLHF con foco en seguridad.

Fine-tuning

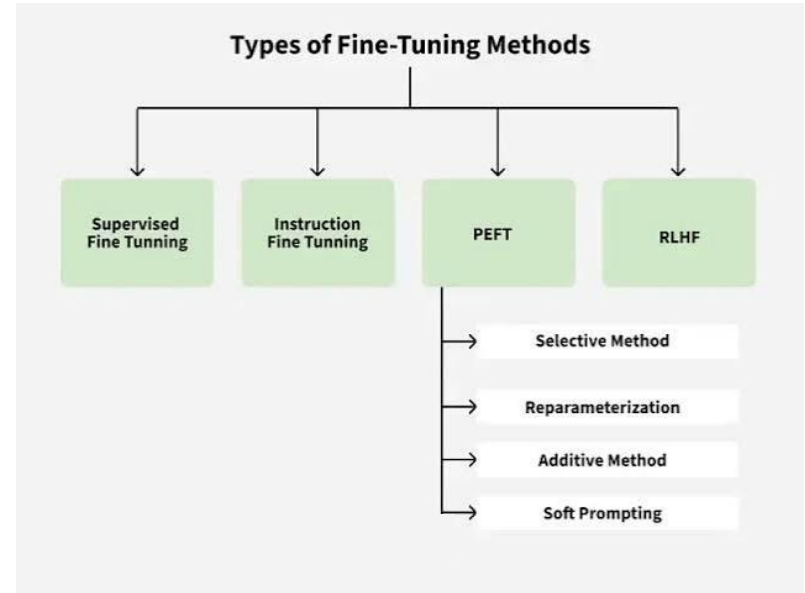
Fine-tuning

- El fine-tuning adapta un modelo a tareas específicas modificando total o parcialmente sus pesos.
- A diferencia de los métodos basados en prompts, que no alteran los pesos, permite mejorar capacidades de dominio, seguridad y, especialmente, el seguimiento de instrucciones, estilo y formato.
- Pero... requiere mucha más memoria y recursos computacionales, por lo que reducir el consumo de memoria es un objetivo clave de las técnicas modernas de fine-tuning.



Fine-tuning

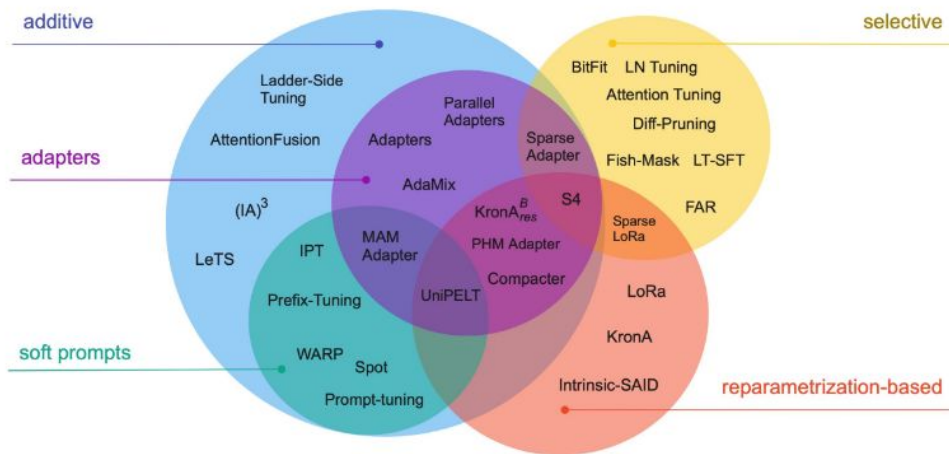
- Hay diferentes técnicas de fine-tuning, nos enfocaremos en técnicas de fine-tuning eficientes en memoria, centradas principalmente en el fine-tuning eficiente en parámetros.
- Cuanta más memoria requiere el fine-tuning, menos accesible es, por eso reducir ese consumo hace la técnica más accesible y permite que más personas adapten modelos a sus propias aplicaciones.



PEFT (Parameter-Efficient Fine-Tuning)

PEFT

- PEFT es una familia de técnicas cuyo objetivo común es adaptar un modelo grande preentrenado a una nueva tarea actualizando solo una fracción pequeña de sus parámetros, dejando el resto congelado.
- Se pueden agrupar en tres grandes familias:



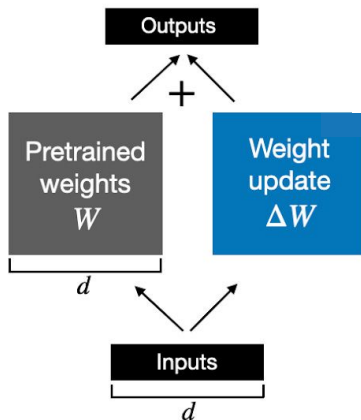
- **Additive:** se agregan parámetros nuevos (pequeños) al modelo, el resto queda congelado. Adapters, Prefix-Tuning, Prompt Tuning.
- **Reparametrization-based:** se reparametriza la actualización de pesos en una forma de bajo rango. LoRA, QLoRA.
- **Selective:** se entrena solo un subconjunto de los parámetros ya existentes del modelo. BitFit, entrenamiento selectivo de capas

LoRA y QLoRA

Idea

- **El problema del fine-tuning:** un modelo de 7B parámetros en full fine-tuning es inviable por su arquitectura (los pesos ocupan aproximadamente 14 GB en fp16, las gradientes también aproximadamente 14 GB, etc)
- Con modelos de 13B, 70B o más, full fine-tuning requiere clústeres multi-GPU costosos.

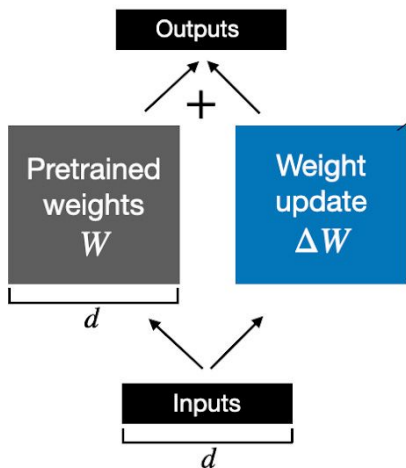
Weight update in **regular finetuning**



LoRA

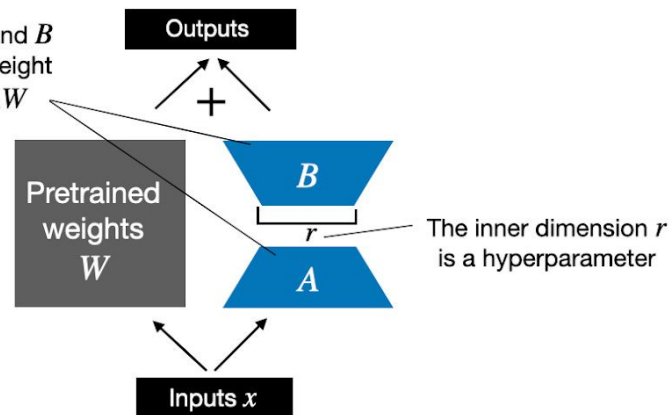
- El cambio de pesos necesario para adaptar un modelo a una nueva tarea tiene rank bajo (low-rank), no hace falta modificar la matriz de pesos completa, alcanza con una actualización de baja dimensionalidad.
- En vez de actualizar la matriz de pesos W directamente, se aprende una descomposición:

Weight update in **regular finetuning**



LoRA matrices A and B approximate the weight update matrix ΔW

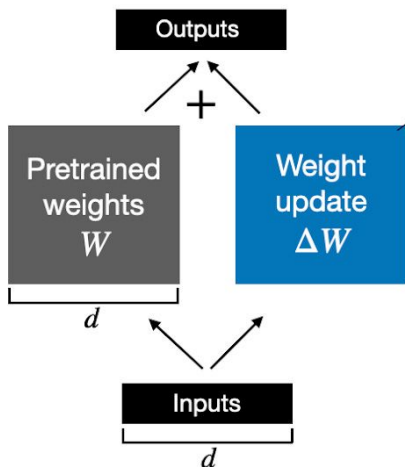
Weight update in **LoRA**



LoRA

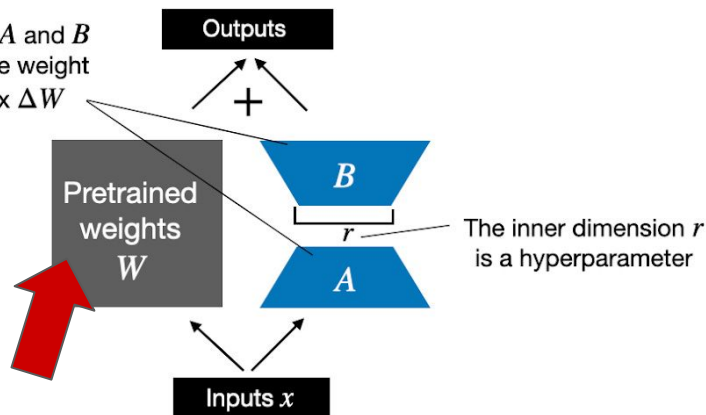
- LoRA reduce los parámetros entrenables, pero el modelo base sigue cargado completo en memoria (en fp16 o bf16).
- Para modelos grandes (30B, 65B, 70B+) esto sigue requiriendo GPUs de alta gama solo para cargar los pesos congelados.
- ¿se puede cuantizar el modelo base para reducir aún más el uso de memoria, sin perder demasiada calidad?: QLoRA

Weight update in **regular finetuning**



Weight update in **LoRA**

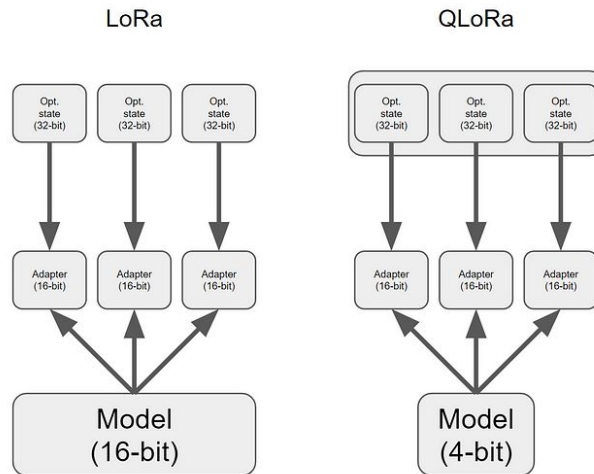
LoRA matrices A and B approximate the weight update matrix ΔW



QLoRA

QLoRA combina LoRA con cuantización para reducir el uso de memoria durante el fine-tuning:

- **Cuantización a 4 bits (NF4):** el modelo base se carga en 4 bits y permanece congelado.
- **Double Quantization:** también cuantiza los factores de escala para ahorrar memoria adicional.
- **Paged Optimizers:** gestiona los picos de memoria para evitar errores Out of Memory.
- **LoRA:** añade pequeños adapters que se entrenan en mayor precisión (FP16/BF16).
- Durante el entrenamiento, los pesos de 4 bits se descuantizan temporalmente cuando son necesarios para los cálculos.





Ejercicio

Instruction Tuning

LINK